

Basi di dati

Maurizio Lenzerini

Danilo Di Primio

Università degli studi La Sapienza

Ingegneria Informatica e Automatica - 3 anno

Gennaio - Dicembre 2026-27

Indice

1	Modello Relazionale	2
1.1	Relazione matematica	2
	Notazioni	3
	Schema di relazione e tupla etichettata	3
	Schemi e istanze	4
1.2	Algebra relazionale	4
	Operatori insiemistici	4
	Ridenominazione	5
	Selezione	6
	Proiezione	7
	Selezione e Proiezione	8
	La negazione in algebra relazionale	8
1.3	Join	9
	Join naturale	9
	Join completo	10
	Theta-join	10
	Equi-join	10
	Self-join	10
	Riassunto join	11
	Esercizi query	11
2	Lessico e Logica	12
2.1	Non	12
2.2	Solo	12
3	SQL	13
3.1	Istruzioni	13
	Select	14
3.2	Ridenominazione As	15
3.3	Distinct, <, *, where, null	15
	Distinct	15
	<	15
	*	15
	Condizione while	15
	gestione dei valori nulli - "is null" e "is not null"	16
3.4	Espressioni nella target list	16
3.5	Selezione, proiezione e join	16
	Select	16
	Join	17
	Self-join	17

Capitolo 1

Modello Relazionale

1.1 Relazione matematica

Definizione 1.1.1. Siano $D_1, D_2, \dots, D_n$ n insiemi:

Il **prodotto cartesiano** $D_1 \times D_2 \times \dots \times D_n$ è l'insieme di tutte le n -uple ordinate $(d_1, d_2, \dots, d_n)$ tali che $d_1 \in D_1, d_2 \in D_2, \dots, d_n \in D_n$.

Definizione 1.1.2. Una **relazione matematica** sugli insiemi $D_1, D_2, \dots, D_n$ è un **sottoinsieme del prodotto cartesiano** $D_1 \times D_2 \times \dots \times D_n$.

Se R è una relazione matematica sugli insiemi $D_1, D_2, \dots, D_n$ tali insiemi sono detti domini, tale relazione su n domini dice che ha grado n . Il numero di n -uple (righe) di una relazione è la cardinalità della relazione stessa. Esempio:

- $D_1 = \{\text{Conti, Totti, Maradona}\}$
- $D_2 = \{\text{Genoa, Roma}\}$
- prodotto cartesiano $D_1 \times D_2$

Conti	Genoa
Conti	Roma
Totti	Genoa
Totti	Roma
Maradona	Genoa
Maradona	Roma

- una relazione: $\text{HaGiocato} \subseteq D_1 \times D_2$

Conti	Roma
Conti	Genoa
Totti	Roma

La relazione matematica è quindi un insieme di ennuple o tuple ordinate su $D_1, D_2, \dots, D_n$, dove ogni tupla è una sequenza di valori della forma $(d_1, \dots, d_n)$. Ciascuna tupla è ordinata nel senso che il valore del primo elemento della tupla proviene dal primo dominio, ma non è definito un ordinamento tra le tuple e sono tutte distinte.

Una relazione del modello relazione si può quindi rappresentare come una tabella in cui gli attributi sono usati come intestazioni delle colonne. La struttura non è posizionale cioè l'ordinamento delle colonne nella tabella è irrilevante. Ogni riga della tabella corrisponde ad una tupla della relazione, e siccome la relazione è un insieme di tuple, non ci possono essere due righe uguali nella tabella.

Partita	Casa	Fuori	RetiCasa	RetiFuori
	'Juve'	'Lazio'	3	1
	'Lazio'	'Milan'	2	0
	'Juve'	'Roma'	1	2
	'Roma'	'Milan'	0	1

Gli attributi devono essere atomici cioè non devono essere un array ma un singolo valore, e non ci devono essere due tuple uguali.

Notazioni

Sia X l'insieme degli attributi di una relazione R . Se t è una tupla di R , cioè una tupla su X , e se $A \in X$ è un attributo in X , allora $t[A]$ oppure anche $t.A$ indica il valore che la tupla t ha in corrispondenza dell'attributo A .
 esempio: $t[Fuori] = Lazio$ $t[Fuori, RetiFuori] = \langle Lazio, 1 \rangle$ $t[] = \text{tupla vuota}$.

Partita	Casa	Fuori	RetiCasa	RetiFuori
t1 →	'Juve'	'Lazio'	3	1
t2 →	'Lazio'	'Milan'	2	0
t3 →	'Juve'	'Roma'	1	2
	'Roma'	'Milan'	0	1

$t1[Fuori] = \text{'Lazio'}$

$t2[RetiCasa] = 2$

$t3[Casa, RetiCasa] = \langle \text{'Juve'}, 1 \rangle$

Schema di relazione e tupla etichettata

La specifica del nome R di una relazione, degli attributi $A_1, A_2, \dots, A_n$ e dei domini di tali attributi $D_1, D_2, \dots, D_n$ forma il cosiddetto schema di relazione, che si denota come $R(A_1 : D_1, A_2 : D_2, \dots, A_n : D_n)$ di conseguenza una tupla di R si può denotare anche come $\langle A_1 : a, A_2 : b, \dots, A_n : c \rangle$

Una tupla di una relazione si può rappresentare come **tupla etichettata**, in cui le etichette sono gli attributi della relazione, ed i valori associati alle etichette sono i valori in corrispondenza dei vari attributi.

Esempio su tabella di prima:

tupla etichettata $t1 = \langle Casa : \text{'Juve'}, Fuori : \text{'Lazio'}, RetiCasa : 3, RetiFuori : 1 \rangle$

Studente	Matricola	Cognome	Nome	Data di nascita
	6554	Rossi	Mario	05/12/1978
	8765	Neri	Paolo	03/11/1976
	9283	Verdi	Luisa	12/11/1979
	3456	Rossi	Maria	01/02/1978

Esame	Studente	Voto	Corso
	3456	30	04
	3456	24	02
	9283	28	01
	6554	26	01

Corso	Codice	Titolo	Docente
	01	Analisi	Mario
	02	Chimica	Bruni
	04	Chimica	Verdi

collegamento tra le due tabelle rappresentato dallo stesso valore presente nelle due tabelle

Schemi e istanze

Definizione 1.1.3. Schema di relazione: un nome di relazione R con un insieme non vuoto di attributi $A_1, \dots, A_n$ ed eventualmente anche i corrispondenti domini $R(A_1, \dots, A_n)$

Definizione 1.1.4. Istanza di relazione su uno schema $R(X)$: insieme r delle tuple sull'insieme di attributi X

Definizione 1.1.5. Relazione: schema T di relazione + istanza su T

Definizione 1.1.6. Schema di base di dati: insieme non vuoto di schemi di relazione $R = R_1(A_1 A_2 \dots A_n), \dots, R_m(B_1 B_2 \dots B_p)$

Definizione 1.1.7. Istanza di base di dati su uno schema $R = \{R_1(X_1), \dots, R_m(X_m)\}$: insieme di relazioni $r = \{r_1, \dots, r_m\}$, dove per ogni $i = 1, \dots, m$, r_i è una istanza relazione sullo schema $R_i(X_i)$

Definizione 1.1.8. Base di dati: schema S di basi di dati + istanza di basi di dati su S

1.2 Algebra relazionale

I suoi operatori sono definiti su relazioni, producono relazioni e possono essere composti.

Operatori dell'algebra relazionale:

- **insiemistici:** unione, intersezione, differenza
- **ridenominazione**
- **selezione**
- **proiezione**
- **join in diverse versioni:** join naturale, prodotto cartesiano, theta-join

Operatori insiemistici

Le relazioni sono insiemi di tuple quindi è sensato definire per esse gli operandi insiemistici, sono operatori binari e i risultati dell'applicazione di tali operandi sono a loro volta relazioni.

Si può applicare **unione**, **intersezione**, **differenza** solo a relazioni che hanno uguali definizioni intensionali

Unione

Laureato

Matricola	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Dirigente

Matricola	Nome	Età
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Laureato $\cup$ Dirigente

Matricola	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45
9297	Neri	33

schema dell'un

tuple da Laure

tuple da Dirige

Intersezione

Laureato

Matricola	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Dirigente

Matricola	Nome	Età
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Laureato $\cap$ Dirigente

Matricola	Nome	Età
7432	Neri	54
9824	Verdi	45

schema
dell'intersezione

tuple da Laureato
e Dirigente

Differenza

Laureato

Matricola	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Dirigente

Matricola	Nome	Età
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Laureato – Dirigente

Matricola	Nome	Età
7274	Rossi	42

schema
della differenza

tuple di Laureato

Ridenominazione

Un'unione sensata ma impossibile in algebra: vogliamo i genitori ed i loro figli

Paternità

Padre	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

Maternità

Madre	Figlio
Eva	Abele
Eva	Set
Sara	Isacco

Paternità $\cup$ Maternità

??

impossibile
eseguire l'unione:
le due relazioni
hanno differenti
insiemi di attributi

modifica lo schema cambiando il nome di uno o più attributi lasciando inalterata l'istanza dell'operando

Paternità

Padre	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

REN_{Genitore ← Padre} (Paternità)

Genitore	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

ridenomina l'attributo
"Padre" in "Genitore"

Ridenominazione: sintassi e semantica

Sintassi:

$$REN_{A_1, A_2, \dots, A_n \leftarrow B_1, B_2, \dots, B_n} (Operando)$$

Semantica:

Lo schema della relazione rappresentata da "Operando" viene modificato sostituendo al nome di attributo B_1 il nome di A_1 , al nome di attributo B_2 il nome A_2 , ..., e al nome di attributo B_n il nome di A_n

Paternità

Padre	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

REN_{Genitore ← Padre} (Paternità)

Genitore	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

REN_{Genitore ← Padre} (Paternità)



REN_{Genitore ← Madre} (Maternità)

Maternità

Madre	Figlio
Eva	Abele
Eva	Set
Sara	Isacco

REN_{Genitore ← Madre} (Maternità)

Genitore	Figlio
Eva	Abele
Eva	Set
Sara	Isacco

Genitore	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco
Eva	Abele
Eva	Set
Sara	Isacco

Selezione

Operatore monadico (un solo argomento) che prevede la formulazione di condizione, produce un risultato che:

- ha lo stesso schema dell'operando
- contiene un sottoinsieme delle tuple dell'operando, quelle che soddisfano la condizione espressamente indicata nell'operatore

Impiegato

Matricola	Cognome	Filiale	Stipendio
7309	Rossi	Roma	55
5998	Neri	Milano	64
9553	Milano	Milano	44
5698	Neri	Napoli	64

impiegati che

- guadagnano più di 50
- guadagnano più di 50 e lavorano a Milano

- hanno lo stesso nome della filiale presso cui lavorano

Selezione: sintassi e semantica

Sintassi:

$$SEL_{Condizione}(Operando)$$

Condizione: espressione booleana in cui gli operandi utilizzano gli attributi delle relazioni in gioco

Semantica:

La relazione risultato ha gli stessi attributi dell'operando e contiene le tuple dell'operando che soddisfano la condizione specificata

esempio: impiegati che guadagnano più di 50

Impiegato

Matricola	Cognome	Filiale	Stipendio
7309	Rossi	Roma	55
5998	Neri	Milano	64
5698	Neri	Napoli	64

risultato

non soddisfa
la condizione

$$SEL_{Stipendio > 50}(Impiegato)$$

impiegati che guadagnano più di 50 e lavorano a Milano

Impiegato

Matricola	Cognome	Filiale	Stipendio
5998	Neri	Milano	64

$$SEL_{Stipendio > 50 \text{ AND } Filiale = 'Milano'}(Impiegato)$$

Proiezione

Operatore monadico, che produce un risultato che:

- ha parte degli attributi dell'operando
- contiene tuple cui contribuiscono tutte le tuple dell'operando: sono le tuple ottenute dall'operando eliminando gli attributi che non compaiono nella lista espressamente indicata nell'operatore

Impiegato

Matricola	Cognome	Filiale	Stipendio
7309	Rossi	Roma	55
5998	Neri	Milano	64
9553	Milano	Milano	44
5698	Neri	Napoli	64

per tutti gli impiegati:

- matricola e cognome
- cognome e filiale

Proiezione: sintassi e semantica

Sintassi:

$$PROJ_{Lista.Attributi}(Operando)$$

Semantica:

la relazione risultato ha i soli attributi contenuti in ListaAttributi, e contiene le tuple ottenute da tutte le tuple dell'operando ristrette agli attributi nella lista

matricola e cognome di tutti gli impiegati

Impiegato

Matricola	Cognome
7309	Neri
5998	Neri
9553	Rossi
5698	Rossi

$$PROJ_{Matricola,Cognome}(Impiegato)$$

Selezione e Proiezione

Combinando selezione e proiezione, possiamo estrarre interessanti informazioni da una relazione

Restituire matricola e cognome degli impiegati che guadagnano più di 50

Impiegato

Matricola	Cognome
7309	Rossi
5998	Neri
5698	Neri

$$PORJ_{Matricola,Cognome}(SEL_{Stipendio>50}(Impiegato))$$

La negazione in algebra relazionale

Per esprimere nel linguaggio dell'algebra condizioni di negazione è utilizzare l'operatore di differenza. Il corrispettivo logico di questo principio è la seguente regola per scrivere query corrette:

l'insieme degli elementi di A che non soddisfano la proprietà B è uguale all'insieme che otteniamo da A togliendo gli elementi che soddisfano la proprietà B:

$$\{x|x \in A \wedge \neg B(x) = A - \{x|B(x)\}$$

esempio:

Concerto(idconc, idart, anno, città)

Artista(idart, nome, tipo)

Evento(ideve, idconc, voto)

richiesta: calcolare l'idconc dei concerti che non sono stati svolti nell'ambito di un evento

$$PROJ_{idconc}(Concerto) - PROJ_{idconc}(Evento)$$

1.3 Join

Combinando selezione e proiezione, possiamo estrarre informazioni da una relazione, non possiamo però correlare informazioni presenti in relazioni diverse. Esistono diversi tipi di join: join naturale, theta-join, equi-join, join esterno ecc.

Join naturale

Operatore binario, produce un risultato:

- il cui schema ha l'unione degli attributi degli operandi
- le cui tuple sono costruite ciascuna a partire da due tuple che si combinano, una di un operando ed una di un altro operando

Join naturale: sintassi e semantica

Siano $R_1(X_1), R_2(X_2)$ con X_1 e X_2 che sono due insiemi, $R_1 \text{ JOIN } R_2$ è una relazione su $X_1 X_2$ il cui insieme di tuple è:

$\{t \text{ su } X_1 X_2 \mid \text{esistono due tuple } t_1 \in R_1 \text{ e } t_2 \in R_2 \text{ tali che } t[X_1] = t_1 \text{ e } t[X_2] = t_2\}$

$R_1 \text{ JOIN } R_2$ è una relazione il cui insieme di tuple contiene ogni tupla t tale che esistono due tuple $t_1 \in R_1$ e $t_2 \in R_2$ tali che

- t negli attributi X_1 ha gli stessi valori di t_1
- t negli attributi X_2 ha gli stessi valori di t_2

Una tupla t esiste solo se le due tuple t_1 e t_2 hanno gli stessi valori negli attributi che R_1 e R_2 hanno in comune. Diciamo che $t_1 \in R_1$ e $t_2 \in R_2$ sono combinabili dal join naturale se $t_1[X_1 \cap X_2] = t_2[X_1 \cap X_2]$. Dalla definizione segue che ogni tupla nel join tra R_1 ed R_2 proviene da due tuple combinabili dal join. In effetti $t_1 \in R_1$ e $t_2 \in R_2$ tali che $t[X_1 \cap X_2] = t_2[X_1 \cap X_2]$ si combinano per ottenere la tupla t tale che $t[X_1] = t_1$ e $t[X_2] = t_2$

Insegna		Join naturale: esempio		Tratta	
Docente	Corso	Corso	Ling		
1	BD	BD	SQL		
2	PS	BD	Java		
3	Reti	PS	Java		
1	PS	PS	UML		

Insegna join Tratta

L'insieme degli attributi del risultato è
 $\{Docente, Corso, Ling\}$
 $=$
 $\{Docente, Corso\} \cup \{Corso, Ling\}$

Docente	Corso	Ling
1	BD	SQL
1	BD	Java
1	PS	Java
1	PS	UML
2	PS	Java
2	PS	UML

nella tabella con il join non è presente il corso Reti perchè non è in comune

Join completo

Un join completo è un join in cui ogni tupla contribuisce al risultato.

Impiegato	Reparto
Rossi	A
Neri	B
Bianchi	B

Reparto	Capo
A	Mori
B	Bruni

Impiegato	Reparto	Capo
Rossi	A	Mori
Neri	B	Bruni
Bianchi	B	Bruni

Join il cui risultato è l'insieme vuoto di tuple

Impiegato	Reparto
Rossi	A
Neri	B
Bianchi	B

Reparto	Capo
D	Mori
C	Bruni

Impiegato	Reparto	Capo

Theta-join

Siano R_1 ed R_2 due relazioni senza attributi in comune tali quindi che il loro join naturale corrisponda al prodotto cartesiano. Ma nella pratica il prodotto cartesiano ha senso solo se combinato con una relazione:

$$SEL_{condizione}(R_1 \text{ JOIN } R_2)$$

La combinazione delle due operazioni viene chiamata **theta-join**, richiede appunto che R_1 ed R_2 non abbiano attributi in comune e viene indicata con

$$R_1 \text{ JOIN}_{condizione} R_2$$

Equi-join

Se il solo operatore di confronto usato nella condizione del Theta-join è l'uguaglianza (=), allora si parla di **equi-join**

Self-join

Il **self-join** è un equi-join in cui la stessa relazione compare sia come operando sinistro sia come operando destro, ovvero, quando una operazione di join coinvolge la stessa relazione due volte. Siccome l'equi-join prevede che i due operandi non abbiano attributi in comune, quando si applica un self-join in uno dei due operandi occorre ridenominare tutti gli attributi della relazione.

Riassunto join

- **Join naturale** tra R_1 e R_2 (si può eseguire sempre)
 - due tuple si accoppiano se e solo se hanno gli stessi valori negli attributi che R_1 ed R_2 hanno in comune
 - se R_1 ed R_2 non hanno attributi in comune, il risultato del join naturale è il prodotto cartesiano tra due relazioni perchè ogni tupla di R_1 si combina con ogni tupla di R_2 per dare origine ad una tupla nel risultato
- **Theta-join** tra R_1 e R_2 (richiede che R_1 e R_2 non abbiano attributi in comune)
 - due tuple si accoppiano se la condizione di join è soddisfatta dalle due tuple
- **Equi-join** tra R_1 e R_2 (richiede che R_1 e R_2 non abbiano attributi in comune)
 - è un theta-join in cui la condizione di join è una uguaglianza tra un attributo di R_1 ed un attributo di R_2

Esercizi query

Tutte le seguenti query sono effettuate sul seguente database:

Concerto(idconc, idart, anno, città)

Artista(idart, nome, tipo)

Evento(ideve, idconc, voto)

query1: trovare nome e tipo degli artisti che hanno tenuto almeno un concerto prima del 2000

soluzione: nome e tipo degli artisti li troviamo nella relazione Artista, mentre i concerti dell'artista li troviamo nella relazione Concerto. Ne segue che per rispondere alla richiesta, la query deve esplorare il legame tra Artista e Concerto e questo si realizza attraverso il join naturale tra le sue due relazioni.

$$PROJ_{nome, tipo}(Artista \text{ JOIN } SEL_{anno < 2000}(Concerto))$$

query2: trovare l'id e nome degli artisti che hanno tenuto almeno un concerto svoltosi in una città diversa da Roma nell'ambito di un evento nel quale ha ottenuto un voto maggiore di 4

soluzione: eseguo due join naturali, il primo tra Artista e Concerto su attributo comune idart, il secondo tra Concerto e Evento su attributo comune idconc.

$$PROJ_{idart, nome}((Artista \text{ JOIN } SEL_{città <> 'Roma'}(Concerto)) \text{ JOIN } SEL_{voto > 4}(Evento))$$

query3: trovare l'id degli artisti che hanno tenuto almeno due concerti nella stessa città in anni diversi.

soluzione: poichè dobbiamo tentare di collegare due tuple della stessa relazione (due concerti tenuti nella stessa città), si deve calcolare il join di Concerto con se stesso, ovviamente stando attenti alle ridenominazioni

$$PROJ_{idart}(REN_{i2 \leftarrow idconc, t1 \leftarrow idart, a1 \leftarrow anno, c1 \leftarrow città, (Concerto) \text{ JOIN }_{idart=t1 \text{ and } città=c1 \text{ and } anno < a1} Concerto))$$

query4: trovare l'id degli artisti che dopo il 2010 hanno tenuto concerti **solo** a Roma

soluzione: dobbiamo capire come intendere "solo", "hanno tenuto concerti solo a Roma" ha lo stesso significato di "non hanno tenuto concerti in una città diversa da Roma".

Quindi per calcolare questa query dobbiamo togliere da tutti gli artisti quelli che dopo il 2010 hanno tenuto almeno un concerto in una città diversa da Roma:

$$PROF_{idart}(Artista) - PROJ_{idart}(SEL_{città <> Roma \text{ and } anno > 2010}(Concerto))$$

query5: trovare le coppie $\langle x, y \rangle$ tali che l'artista con id x non ha mai fatto concerti nella città y .

soluzione: da tutte le possibili coppie $\langle x, y \rangle$ tali che x è l'id di un artista e y è una città, leviamo le coppie $\langle x, y \rangle$ tali che l'artista di id x ha fatto almeno un concerto nella città y :

$$(PROJ_{idart} \text{ JOIN } PROJ_{città}(Concerto)) - PROJ_{idart, città}(Concerto)$$

Capitolo 2

Lessico e Logica

2.1 Non

Quando nelle query troviamo la richiesta **non**, devo ricavare tale richiesta togliendo dal totale delle tuple tutte le tuple che soddisfano la negazione della condizione.

Esempio:

concerto (idconc, idart, anno, città)

Artista (idart, nome, tipo)

Evento (ideve, idconc, voto)

query: Calcolare l'idconc dei concerti che non sono stati svolti nell'ambito di un evento

soluzione: $PROJ_{idconc}(Concerto) - PROJ_{idconc}(Evento)$

2.2 Solo

Quando nelle query troviamo la richiesta **solo**, dobbiamo ricondurla alla negazione **non**

Esempio (stesso database dell'esempio precedente):

query: Trovare l'id degli artisti che dopo il 2010 hanno tenuto concerti solo a Roma

soluzione: "hanno tenuto concerti solo a Roma" ha lo stesso significato di "non hanno tenuto concerti in una città diversa da Roma"

$PROJ_{idart}(Artista) - PROJ_{idart}(SEL_{città \neq Roma \text{ and } anno > 2010}(Concerto))$

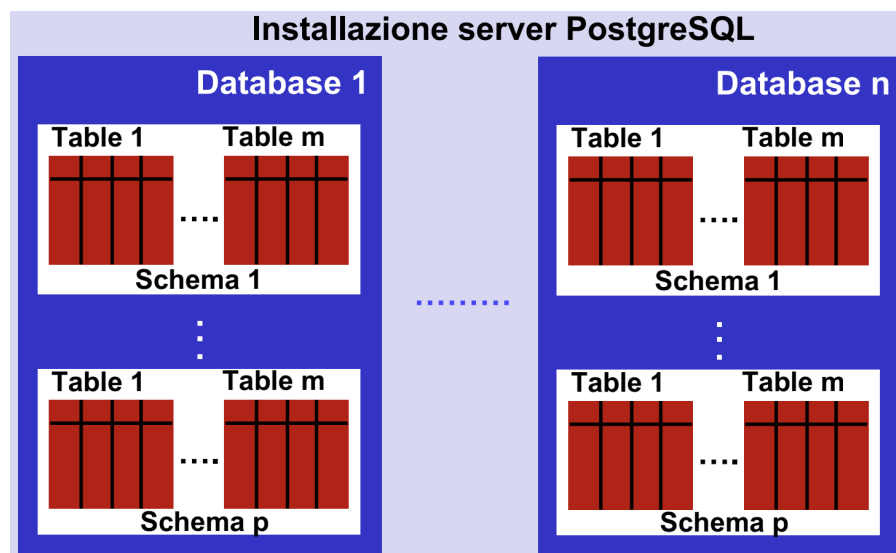
Capitolo 3

SQL

Structure Query Language è un linguaggio con varie funzionalità, che contiene:

- DDL (data definition language)
- DML (data manipulation language)

Una tabella in SQL è definita come un multi insieme di tuple, in particolare, se una tabella non ha una primary key o un insieme di attributi definiti come unique, allora potranno comparire due tuple uguali nella tabella; ne segue che una tabella SQL non è in generale una relazione.



Ogni utente si collega con un database, che si denota con Schema1.Table1 (oppure con solo Table1 se Schema1 è implicito), un utente connesso con Database1 "vede" tutti gli schemi di Database1 e nessuno schema di un altro Database. Un Database Postgres è una collezione di basi di dati, ognuno con il relativo schema e la relativa istanza.

3.1 Istruzioni

- Ogni **tabella** si denota con NomeSchema.NomeTabella
- Ogni **attributo** di una tabella si denota con NomeSchema.NomeTabella.Attributo

Quando l'ambiguità sullo schema non sussiste si può omettere NomeSchema. e scrivere semplicemente NomeTabella
Quando l'ambiguità sullo schema non sussiste si può omettere NomeSchema. e scrivere NomeTabella.Attributo
Quando l'ambiguità sulla tabella non sussiste si può omettere NomeTabella. e scrivere semplicemente Attributo

Select

L'istruzione di interrogazione in SQL è **select** che definisce una interrogazione (query) e restituisce il risultato della valutazione di quella query sulla base di dati in forma di tabella. La sua forma elementare è:

```
select Attributo,...,Attributo
from   Tabella
where  Condizione
```

semantica:

ogni tupla t della Tabella è indicata nella clausola from viene analizzata. Se t non soddisfa la condizione nella clausola where, allora viene ignorata. Altrimenti da t viene prodotta la target list (lista attributi) secondo quanto specificato nella target list che appare dopo select e la tupla risultante da tale target list viene inserita nel risultato. Il risultato della esecuzione della query viene restituito nel canale di output del sistema e volendo memorizzarlo nella base di dati. La semantica descritta chiarisce che l'istruzione è analoga alla seguente espressione dell'algebra relazionale

$$PROJ_{Attributo,...,Attributo}(SEL_{Condizione}(Tabella))$$

Diciamo analoga e non equivalente perchè in sql la tabella può avere duplicati.

Esempio:

maternita

madre	figlio
Luisa	Maria
Luisa	Luigi
Anna	Olga
Anna	Filippo
Maria	Andrea
Maria	Aldo

paternita

padre	figlio
Sergio	Franco
Luigi	Olga
Luigi	Filippo
Franco	Andrea
Franco	Aldo

Lo schema di questa base di dati è S

persone

nome	eta	reddito
Andrea	27	21
Aldo	25	15
Maria	55	42
Anna	50	35
Filippo	26	30
Luigi	50	40
Franco	60	20
Olga	30	41
Sergio	85	35
Luisa	75	87

nelle slides che seguono assumiamo che persone diverse abbiano nomi diversi

Vogliamo il nome e reddito delle persone con meno di 30 anni.

$$PROJ_{nome,reddito}(SEL_{eta<30}(persone))$$

```
select S.persone.nome, S.persone.reddito
from S.persone
where S.persone.eta<30
```

nome	reddito
Andrea	21
Aldo	15
Filippo	30

Da ora in poi assumiamo di essere nell'ambito dello schema S e quindi possiamo omettere il nome dello schema. In questo caso possiamo quindi scrivere:

```
select persone.nome, persone.reddito
from persone
where persone.eta<30
```

Nella query non c'è ambiguità su quali sono gli attributi: essi sono certamente quelli della tabella "persone" quindi la query si può scrivere:

```
select nome, reddito
from persone
where eta<30
```

3.2 Ridenominazione As

"as" nella lista degli attributi serve a ridenominare gli attributi, specificando esplicitamente un nome per un attributo del risultato.

Esempio:

```
select nome as name, reddito as salary
from persone
where eta<30
```

restituisce con risultato una tabella con due attributi, il primo nome name ed il secondo salary.

"as" serve anche ad assegnare un nuovo nome alle tabelle nell'ambito di una query.

Esempio:

```
select p.nome as name, p.reddito as salary
from persone as p
where p.eta<30
```

3.3 Distinct, <, *, where, null

Distinct

Per non avere duplicati nella query nella select aggiungiamo distinct

Esempio:

```
select distinct attributo1,...,attributo2 from tabella1
```

<

La utilizziamo per evitare ridondanze quando stiamo confrontando righe della stessa tabella tra loro, tipicamente in un self join, e vogliamo ottenere ogni coppia una sola volta.

Esempio:

```
select p1.nome, p2.nome
from persona p1, persona p2
where p1.id < p2.id
```

*

Quando vogliamo che una query restituisca tutti gli attributi usiamo "*" nella select al posto della lista degli attributi.

Esempio:

Data una tabella R sugli attributi A1,...,An:

```
select *
from tabella1
where condizione
```

equivale a

```
select A1,...,An
from R
where condizione
```

Condizione while

Quando eseguiamo una query in cui non c'è una condizione possiamo scrivere while true o non scrivere proprio where.

Esempio:

```
select A1,...,An
from tabella1
```

equivale a

```
select A1,...,An
from R
where true
```

Nella clausola where possono comparire espressioni booleane qualunque

Esempio:

```
select * from persone where reddito > 25 and (eta < 30 or eta >60)
```

Nelle condizioni nella clausola where si possono usare molti operatori che SQL mette a disposizione.

Ad esempio l'operatore `like` che consente di verificare che una stringa appartenga al linguaggio definito da una espressione regolare.

Esempio:

se vogliamo conoscere quali sono le persone che hanno un nome che inizia per 'A', ha 'd' come terza lettera e può continuare con altri caratteri.

```
select *
from persone
where nome like 'A_d%'
```

gestione dei valori nulli - "is null" e "is not null"

nelle condizioni che compaiono nella clausola where si possono usare anche i predicati "is null" e "is not null" per gestire i valori nulli

Esempio:

vogliamo le persone la cui eta è o potrebbe essere maggiore di 40

```
select *
from persone
where eta > 40 or eta is null
```

3.4 Espressioni nella target list

Ogni elemento della target list può essere una espressione che fa uso dei valori memorizzati negli attributi delle tuple del risultato.

Esempio:

```
select eta, reddito/2
from persone
where nome='Luigi'
```

`reddito/2` → espressione aritmetica che fa uso del valore dell'attributo `reddito` e lo divide per 2

Nella target list può comparire un numero qualunque di espressioni e all'interno di tali espressioni possono ovviamente comparire anche costanti

Esempio:

```
select 'Gigi' as nomePersona, eta, reddito/2 as redditoSemestrale
from persone
where nome='Luigi'
```

Espressione costituita da una costante di tipo stringa che aggiunge un attributo `nomePersona` con la stringa 'Gigi'

nomePersona	età	redditoSemestrale
Gigi	50	20

3.5 Selezione, proiezione e join

le istruzioni `select` che abbiamo visto finora hanno una sola tabella nella clausola `from` e quindi permettono di realizzare:

- selezioni
- proiezioni
- ridenominazioni

I join si possono realizzare indicando due o più tabelle nella clausola `from`, separate da virgola

Select

sintassi e semantica

La forma base delle `select` in SQL è:

```
select <target list> --> lista attributi
from R1,R2,...,Rn
where <condizione>
```

In algebra relazionale: $PROJ_{targetlist}(SEL_{<condizione>}(R_1 \times R_2 \times \dots \times R_n))$

Ogni tupla t del prodotto cartesiano viene analizzata. Se t non verifica la condizione della clausola WHERE, essa viene ignorata. Se invece t verifica la condizione della clausola WHERE, allora da t viene prodotta la target list secondo la proiezione specificata nella clausola SELECT e la tupla risultante da tale target list viene inserita nel risultato

Attenzione: questo non significa che il DBMS calcola il prodotto cartesiano di $R_1, R_2, \dots, R_n$. Significa che il risultato ottenuto è lo stesso di quello che si ottiene partendo dal prodotto cartesiano.

Join

Date due relazioni: $R_1(A_1, A_2)$ e $R_2(A_3, A_4)$

```
select R1.A1, R2.A4
from R1, R2
where R1.A2 = R2.A3
```

è analoga a: $PROJ_{A_1, A_4}(R_1 JOIN_{A_2=A_3} R_2)$

Possono essere necessarie ridenominazioni:

- nella target list
- nella clausola from (in particolare, introdurre alias consente di riferirsi due o più volte alla stessa tabella)

Esempio:

```
select X.A1 as B1, ...
from R1 as X, R2 as Y, R1 as Z
where X.A2 = Y.A3 and Y.A4 = Z.A1
```

Si può anche scrivere senza 'as'.

Date le tabelle $R_1(A_1, A_2)$ e $R_2(A_3, A_4)$

```
select distinct X.A1 as B1, Y.A4 as B2
from R1 as X, R2 as Y, R1 as Z
where X.A2 = Y.A3 and Y.A4 = Z.A1
```

è equivalente a: $REN_{B_1, B_2 \leftarrow A_1, A_4}(PROJ_{A_1, A_4}(SEL_{A_2=A_3 \text{ and } A_4=C_1}(R_1 JOIN R_2 JOIN REN_{C_1, C_2 \leftarrow A_1, A_2}(R_1))))$

Self-join

Il self-join è un join in cui la stessa relazione compare sia come operando sinistro sia come operando destro ed è cruciale quando dobbiamo combinare due tuple della stessa relazione.

Esempio:

`persone(nome, eta, reddito)`

query: Vogliamo le coppie di persone con lo stesso reddito

soluzione:

```
select p1.nome, p1.eta, p1.reddito, p2.nome, p2.eta
from persone p1, persone p2
where p1.reddito=p2.reddito and p1.nome<p2.nome
```

utilizziamo $p1.nome < p2.nome$ per eliminare le tuple ridondanti, in quanto lascia solo le tuple in cui $p1.nome$ viene prima in ordine alfabetico di $p2.nome$